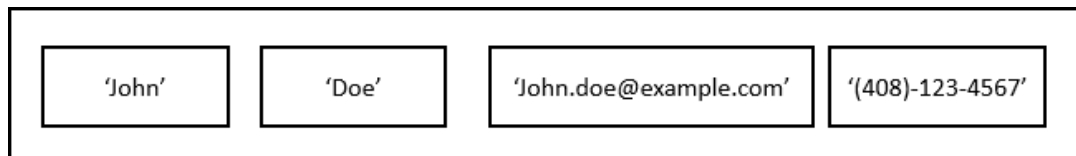


Unit - 6 Records and Procedures [5 Hrs.]

PL/SQL Record

A PL/SQL record is a composite data structure which consists of multiple fields; each has its own value.

The following picture shows an example record that includes first name, last name, email, and phone number:



PL/SQL record helps you simplify your code by shifting from field-level to record-level operations.

Records consist of different fields, similar to a row of a database table.

For example, you want to keep track of your books in a library. You might want to track the following attributes about each book, such as Title, Author, Subject, Book ID. A record containing a field for each of these items allows treating a BOOK as a logical unit and allows you to organize and represent its information in a better way.

Types of Record

PL/SQL can handle the following types of records –

- Table-based
- Cursor-based records
- User-defined records

Before using a record, you must declare it in a declaration section of a block.

Table-Based Records

To declare a table-based record, you use the %ROWTYPE attribute with a table name. A table-based record has each field corresponding to a column in a table.

Syntax,

```
DECLARE
    record_name table_name%ROWTYPE;
```

Example is shown below:

Let's assume following employee table with data shown below,

EID	NAME	SALARY
101	Ram	25000
102	Shyam	55000
103	Gita	34000

```
DECLARE
    emp_record employee%ROWTYPE;
BEGIN
    SELECT * INTO emp_record
    FROM employee WHERE eid=102;

    --accessing fields from record
    DBMS_OUTPUT.PUT_LINE('Eid: ' || emp_record.eid);
    DBMS_OUTPUT.PUT_LINE('Name: ' || emp_record.name);
    DBMS_OUTPUT.PUT_LINE('Salary: ' || emp_record.salary);
END;
/
```

Output:

Statement processed.

Eid: 102

Name: Shyam

Salary: 55000

Cursor-Based Records

To declare a cursor-based record, you use the %ROWTYPE attribute with a cursor name. A cursor-based record has each field corresponding to a column in a cursor.

Syntax,

```
DECLARE
    record_name cursor_name%ROWTYPE;
```

Example is shown below:

```
DECLARE
    CURSOR mycursor IS
        SELECT * FROM employee;
    emp_record mycursor%ROWTYPE;
BEGIN
    OPEN mycursor;

    LOOP
        FETCH mycursor INTO emp_record;
        EXIT WHEN mycursor%NOTFOUND;
        --accessing fields from record
        DBMS_OUTPUT.PUT_LINE(emp_record.eid || ' ' || emp_record.name);
    END LOOP;

    CLOSE mycursor;
END;
/
```

Output:

Statement processed.
101 Ram
102 Shyam
103 Gita

Record in For Loop Cursor

```
DECLARE
    CURSOR mycursor IS
        SELECT * FROM employee;
BEGIN
    FOR emp_record IN mycursor
    LOOP
        DBMS_OUTPUT.PUT_LINE(emp_record.eid || ' ' || emp_record.name);
    END LOOP;

END;
/
```

User-Defined Records

The table-based and cursor-based records are good enough when you need to create records based on existing structures. **If you want to create a record whose structure is not based on the existing ones, then you use user-defined record.**

To declare a programmer-defined record, you use the following steps:

1. Define a record type that contains the structure you want in your record.
2. Declare a record based on the record type.

The following shows the syntax for defining a record type:

```
TYPE record_type IS RECORD (  
    field_name1 data_type1 [[NOT NULL] := | DEFAULT default_value],  
    field_name2 data_type2 [[NOT NULL] := | DEFAULT default_value],  
    ...  
);
```

To declare a record based on the predefined record type, you use the following syntax:

```
record_name record_type;
```

Let's create a new record that containing fields for student.

```
DECLARE  
TYPE student IS RECORD  
(sid number,  
    name varchar(50),  
    address varchar(100)  
);  
st_record student;
```

Complete Example is shown below:

```
DECLARE  
    TYPE student is RECORD  
    (  
        | sid INT, name VARCHAR(30), address VARCHAR(100)  
    );  
    st_record student;  
BEGIN  
    --assigning values in record  
    st_record.sid:=101;  
    st_record.name:='Ram';  
    st_record.address:='Btm';  
  
    --accessing values  
    DBMS_OUTPUT.PUT_LINE('Sid: ' || st_record.sid);  
    DBMS_OUTPUT.PUT_LINE('Name: ' || st_record.name);  
    DBMS_OUTPUT.PUT_LINE('Address: ' || st_record.address);  
END;  
/
```

Output:

```
Statement processed.  
Sid: 101  
Name: Ram  
Address: Btm
```

Inserting a new row using record:

Let's insert a new row in above **employee** table.

```
DECLARE
    emp_record employee%ROWTYPE;
BEGIN
    --assign values in record
    emp_record.eid:=104;
    emp_record.name:='Raaju Poudel';
    emp_record.salary:=50000;

    --inserting into employee table
    INSERT INTO employee VALUES emp_record;
END;
/
```

After running above code, you can confirm insert operation by selecting data:

```
SELECT * FROM employee;
```

EID	NAME	SALARY
101	Ram	25000
102	Shyam	55000
103	Gita	34000
104	Raaju Poudel	50000

Updating a row using Record:

To update a row from a %ROWTYPE record, you use the **SET ROW keyword** as shown in the following example:

```
DECLARE
    emp_record employee%ROWTYPE;
BEGIN
    --selecting and storing data in record
    SELECT * INTO emp_record FROM employee
    WHERE eid=103;

    --change name to Arun
    emp_record.name:='Arun';

    --update row in table
    UPDATE employee SET ROW=emp_record
    WHERE eid=emp_record.eid;
END;
/
```

After selection you'll get following result:

```
SELECT * FROM employee WHERE eid=103;
```

EID	NAME	SALARY
103	Arun	34000

Nested Record

A record can contain a field which is another record. This type of record is called **nested record**. Nesting records is a powerful way to structure your program data and hide complexity in your code.

The following example declares a **record type** named **address**. Then in the **record type** **customer** has two fields **ship_to** and **bill_to** which are based on the **address record type**.

```
DECLARE
    TYPE address IS RECORD
    (
        city VARCHAR(50),
        district VARCHAR(50)
    );

    TYPE customer IS RECORD
    (
        cid INT,
        name VARCHAR(30),
        ship_to address --derived from another record
    );

    cus_record customer;

BEGIN
    --assigning values
    cus_record.cid:=101;
    cus_record.name:='Ram';
    cus_record.ship_to.city:='Birtamod';
    cus_record.ship_to.district:='Jhapa';

    --accessing values
    DBMS_OUTPUT.PUT_LINE(cus_record.cid || ' ' || cus_record.name || ' '
    || cus_record.ship_to.city || ' ' || cus_record.ship_to.district);
END;
```

Output:

```
Statement processed.
101 Ram Birtamod Jhapa
```

PL/SQL Procedure

The PL/SQL stored procedure or simply a procedure is a PL/SQL block which performs one or more specific tasks. It is just like procedures in other programming languages.

It is a named block.

The procedure contains a header and a body.

- **Header:** The header contains the name of the procedure and the parameters or variables passed to the procedure.
- **Body:** The body contains a declaration section, execution section and exception section similar to a general PL/SQL block.

Syntax for creating procedure:

```
CREATE [OR REPLACE] PROCEDURE procedure_name  
    (parameter_list)  
IS  
    [declaration_section]  
BEGIN  
    executable_section  
    [EXCEPTION  
        exception_section]  
END [procedure_name];
```

Below are the characteristics of Procedure in PL/SQL:

- Procedures are standalone blocks of a program that can be stored in the database.
- Call to these PLSQL procedures can be made by referring to their name, to execute the PL/SQL statements.
- It is mainly used to execute a process in PL/SQL.
- It can have nested blocks, or it can be defined and nested inside the other blocks or packages.
- It contains declaration part (optional), execution part, exception handling part (optional).
- The values can be passed into Oracle procedure or fetched from the procedure through parameters.
- These parameters should be included in the calling statement.
- A Procedure in SQL can have a RETURN statement to return the control to the calling block, but it cannot return any values through the RETURN statement.
- Procedures cannot be called directly from SELECT statements. They can be called from another block or through EXEC keyword.

Passing Parameters in a Procedure

When you want to create a procedure or function, you have to define parameters. There are three ways to pass parameters in a procedure:

1. **IN parameters:** The IN parameter can be referenced by the procedure or function. The value of the parameter **cannot be overwritten by the procedure or the function**.
2. **OUT parameters:** The OUT parameter cannot be referenced by the procedure or function, but the **value of the parameter can be overwritten by the procedure or function**.
3. **IN OUT parameters:** The INOUT parameter can be referenced by the procedure or function and **the value of the parameter can be overwritten by the procedure or function**.

IN Parameter

An IN parameter is read-only. You can reference an IN parameter inside a procedure, but you cannot change its value. Oracle uses IN as the default mode. It means that if you don't specify the mode for a parameter explicitly, Oracle will use the IN mode.

OUT Parameter

An OUT parameter is writable. Typically, you set a returned value for the OUT parameter and return it to the calling program. Note that a procedure ignores the value that you supply for an OUT parameter.

IN OUT Parameter

An INOUT parameter is both readable and writable. The procedure can read and modify it.

Example of Stored Procedure without parameters:

```
--procedure without parameters
CREATE OR REPLACE PROCEDURE myproc
IS
    message VARCHAR(30):='I am inside stored procedure!';
BEGIN
    DBMS_OUTPUT.PUT_LINE(message);
END;
```

To execute above procedure,
EXEC myproc;

Output:

```
Statement processed.
I am inside stored procedure!
```


Stored procedure with IN parameter:

```
CREATE OR REPLACE PROCEDURE myproc(  
  a IN NUMBER, b IN NUMBER  
)  
IS  
  --declarative section  
  c NUMBER;  
BEGIN  
  c:=a+b;  
  DBMS_OUTPUT.PUT_LINE('Sum: ' || c);  
END;
```

Calling above procedure:

```
BEGIN  
  myproc(20,30);  
END;
```

Or

```
EXEC myproc(20,30);
```

Output:

Statement processed.
Sum: 50

Stored procedure with OUT parameter:

```
CREATE OR REPLACE PROCEDURE myproc(  
  a IN NUMBER, b IN NUMBER,  
  res OUT NUMBER  
)  
IS  
BEGIN  
  res:=a+b;  
END;
```

Calling stored procedure,

```
DECLARE  
  result NUMBER;  
BEGIN  
  myproc(20,30,result);  
  DBMS_OUTPUT.PUT_LINE('Result: ' || result);  
END;
```

Output:

Statement processed.
Result: 50

Stored procedure with IN OUT parameter

```
CREATE OR REPLACE PROCEDURE myproc(  
    num1 IN NUMBER,  
    num2 IN OUT NUMBER  
)  
IS  
BEGIN  
    num2:=num1+num2;  
END;
```

Procedure call,

```
DECLARE  
    res NUMBER:=12;  
BEGIN  
    myproc(20,res);  
    DBMS_OUTPUT.PUT_LINE('Result: ' || res);  
END;
```

Output:

Statement processed.
Result: 32

Inserting data in a table using stored procedure:

Step 1: Create a book table as follows:

```
CREATE TABLE book(  
    bid INT,  
    bname VARCHAR(30),  
    author VARCHAR(30)  
);
```

Step 2: Create a stored procedure that inserts new row in book table:

```
CREATE OR REPLACE PROCEDURE myproc(  
    bid INT,  
    bname VARCHAR2,  
    author VARCHAR2  
)  
IS  
BEGIN  
    --insert query  
    INSERT INTO book VALUES(bid,bname,author);  
END;
```

Step 3: Execute stored procedure

```
BEGIN
    --passing values to insert
    myproc(101, 'Java', 'Raaju Poudel');
    DBMS_OUTPUT.PUT_LINE('Inserted Successfully!');
END;
```

Output:

Statement processed.
Inserted Successfully!

To confirm data inserted in table select book table as follows:

```
SELECT * FROM book;
```

BID	BNAME	AUTHOR
101	Java	Raaju Poudel

Updating data in a table using stored procedure:

```
CREATE OR REPLACE PROCEDURE myproc(
    id IN INT,
    name IN VARCHAR2
)
IS
BEGIN
    --update query
    UPDATE book SET bname=name
    WHERE bid=id;
END;
```

Calling a stored procedure,

```
BEGIN
    --passing values to insert
    myproc(101, 'Database Programming');
    DBMS_OUTPUT.PUT_LINE('Updated Successfully!');
END;
```

Select table for confirmation,

```
SELECT * FROM book;
```

BID	BNAME	AUTHOR
101	Database Programming	Raaju Poudel

Deleting data from a table using stored procedure:

```
CREATE OR REPLACE PROCEDURE myproc(  
    id IN INT  
)  
IS  
BEGIN  
    --delete query  
    DELETE FROM book WHERE bid=id;  
END;
```

Calling a stored procedure,

```
BEGIN  
    myproc(101);  
    DBMS_OUTPUT.PUT_LINE('Deleted Successfully!');  
END;
```

Output:

Statement processed.
Deleted Successfully!